

Module – 3

Planning & its Importance

Classical Partial order planning & Conditional Planning

Uncertainty, type of uncertainty

Probabilistic Reasoning-joint distribution reasoning,
Bayesian networks

Learning: explanation-based learning, induction learning-Decision Tree,
statistical learning-Bayesian Learning

Expectation maximization, Hidden Markov model, closed world problems

Dr. Manas Ranjan Patra
NIST University

Planning

- Planning: devising a plan of action to achieve one's goals
- Planning in AI refers to the process of developing a strategy (sequences of actions) that an intelligent agent should follow in order to achieve a specific goal.
- It may involve decision-making, problem-solving, and optimization techniques to reason about future states and strategy to move towards achieving its objectives.

Key Components of Planning in AI

- **Initial State:** The starting point of the AI system/agent.
- **Goal State:** The desired outcome or set of conditions the system aims to achieve.
- **Actions/Operators:** The available moves or decisions the agent can make. Each action transitions the system from one state to another.
- **State Space:** The entire set of possible states.
- **Plan:** A sequence of actions that the agent need to take from the initial state to the goal state.
- **Cost:** Cost associated with actions in terms of time, resources that must be minimized during planning.

Approaches for Planning

- **Classical Planning**

- Involves fully observable, deterministic environments where search techniques such as DFS, BFS, Dijkstra, etc. can be used.

- **Heuristic-Based Planning**

- Uses heuristics to guide the search process while estimating the cost to move from one state to another.

- **Hierarchical Task Networks**

- Breaks a complex problem into smaller, more manageable subtasks in a hierarchical fashion until a subtask can be directly solved.

- **Probabilistic Planning**

- Takes into account uncertainty in outcomes.
- It is useful in environments where actions may not always result in a known or expected state.

- **Temporal Planning**

- Deals with situations when actions have time durations, deadlines, or synchronization constraints.
- Useful in robotics, logistics, scheduling

Classical Partial-Order Planning (POP)

- In total-order planning (TOP), the sequence of actions is fully determined.
- In partial order planning the order of some actions is not completely specified. This provides greater flexibility in plan execution.
- Partial-Order Plans: plans are represented as partial orders of actions, meaning that some actions are ordered relative to others, but not all actions are linearly sequenced. This gives flexibility during plan execution, as actions that are not constrained by ordering dependencies can be performed in any sequence.

Key concepts:

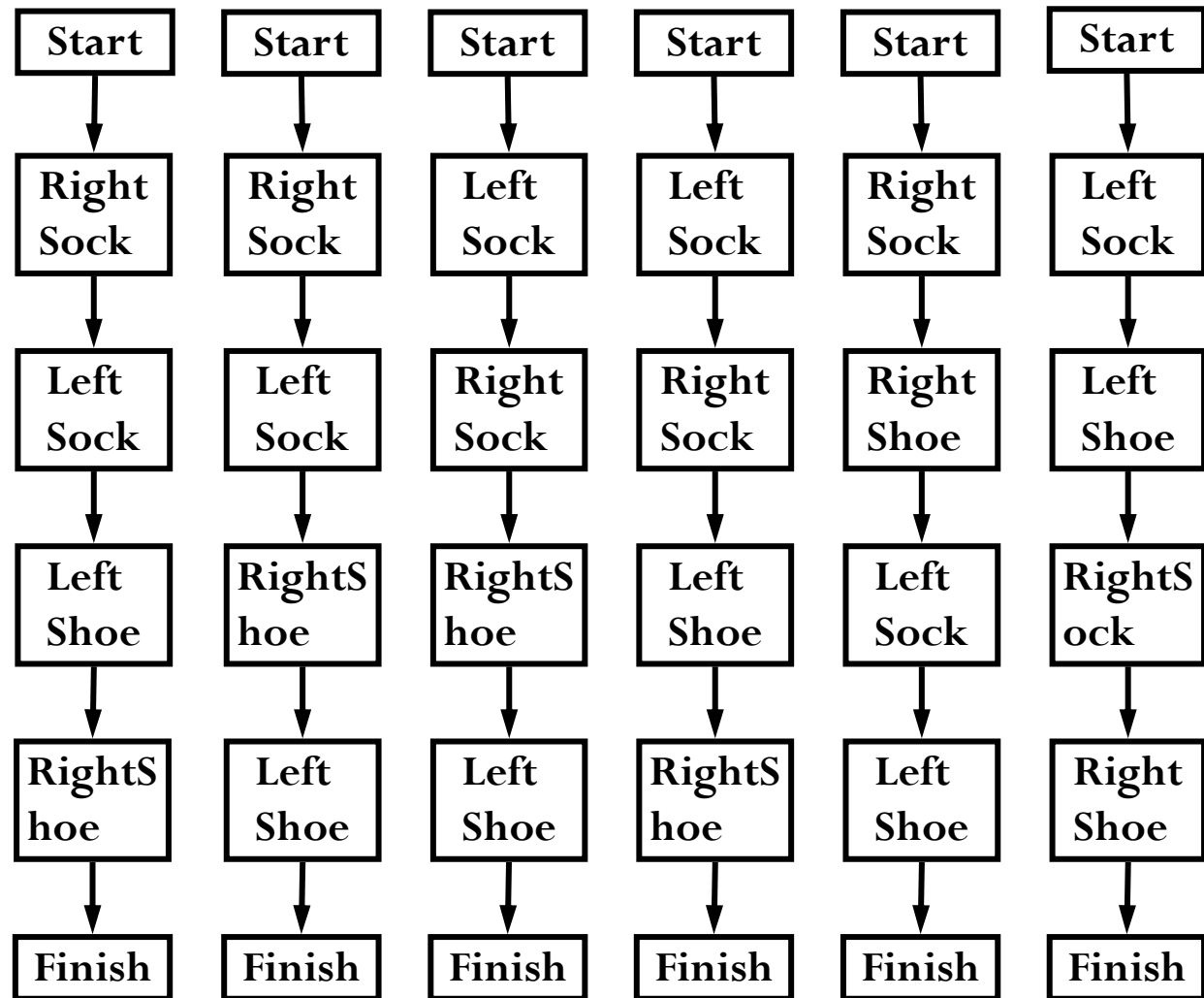
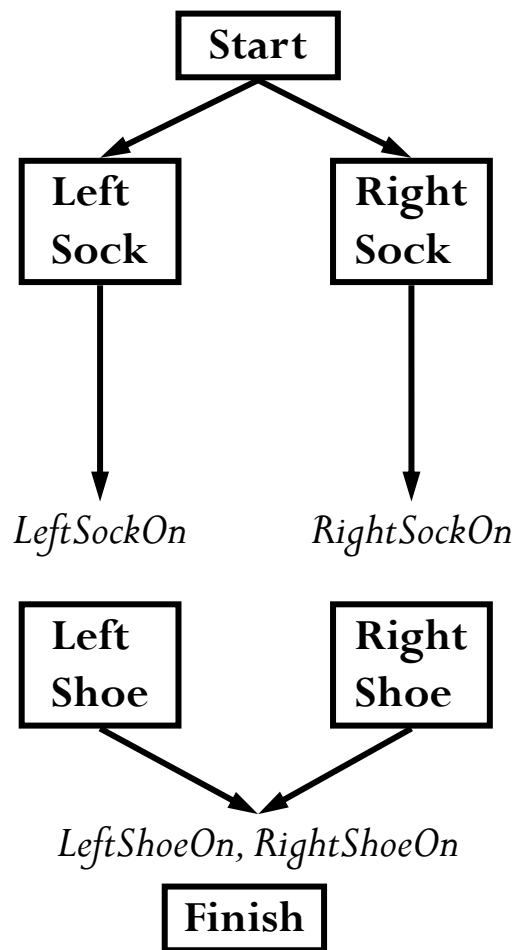
- **Actions (Operators):** Represent tasks that transition the system from one state to another where each action has:
 - Preconditions: Conditions that must be true before the action can be executed.
 - Effects: The changes of state that happens when the action is performed.
- **Causal Links:** These represent the dependencies between actions.

A causal link from action $A_1 \rightarrow A_2$ means the effect of A_1 serves as precondition for A_2
- **Threats:** Occur when an action in the plan threatens to invalidate a causal link by undoing a precondition that another action depends on.
- **Flaw Resolution:** In the planning process, flaws such as missing preconditions or threats to causal links are identified and resolved.

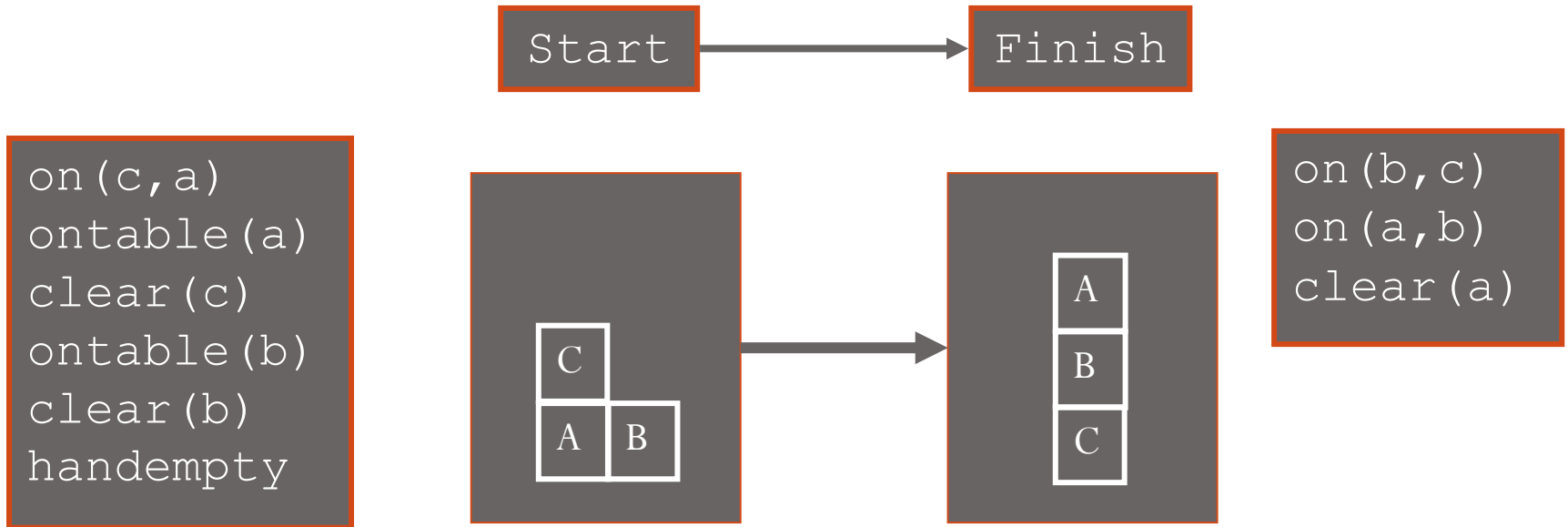
Advantages of Partial-Order Planning

- **Flexibility:** It provides flexibility by not imposing an unnecessary sequence of actions. Different components of the plan can be executed in parallel or in any order, as long as the constraints are satisfied.
- **Efficiency:** Fewer constraints on the order of actions can lead to more efficient execution, as actions can be performed in parallel or rearranged depending on the current situation.
- **Handling Complex Domains:** Partial-order planning is well-suited for complex & dynamic environments where actions can be interleaved or reordered dynamically.

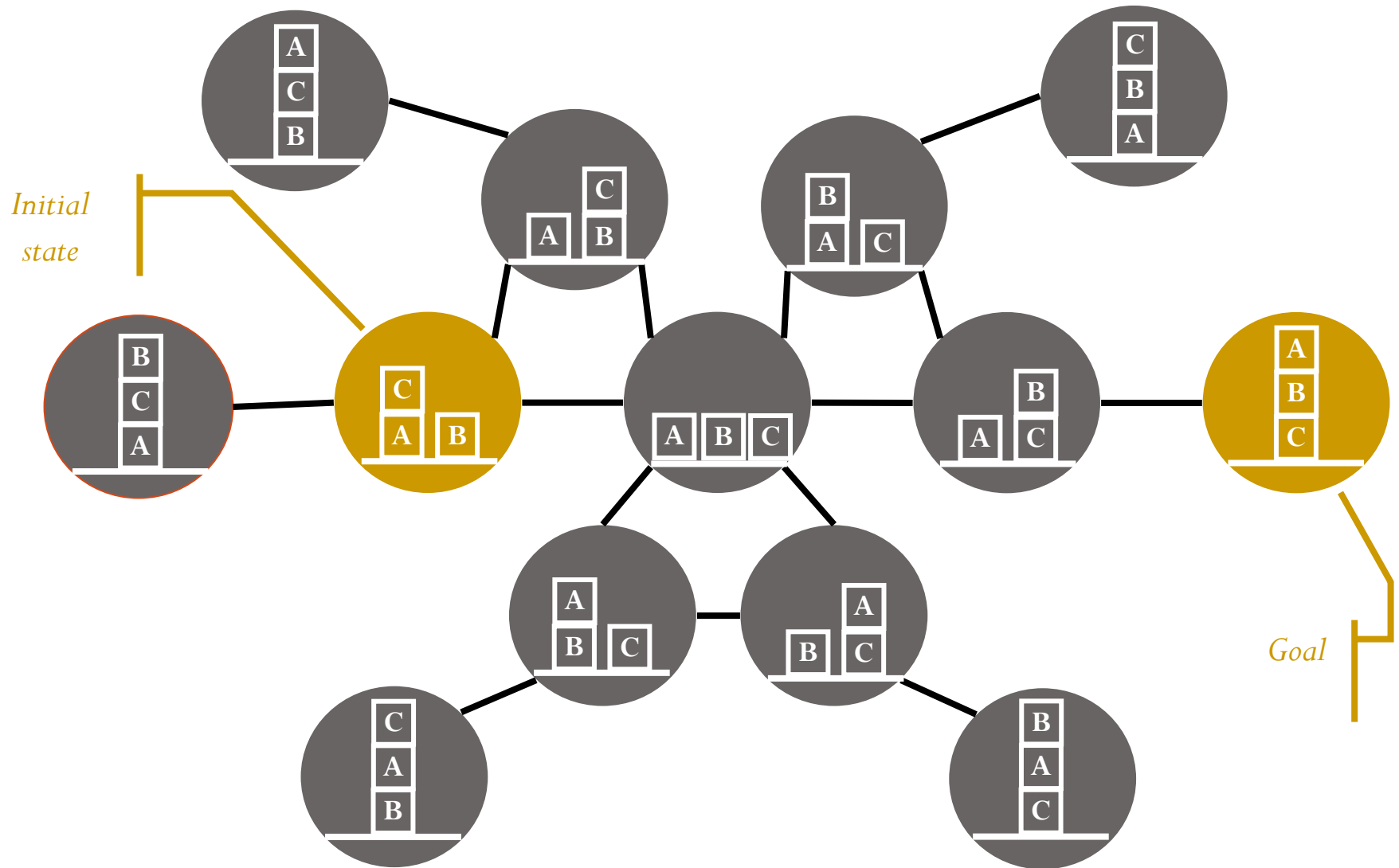
Transformation of Partial Order Plan into Total-order Plan



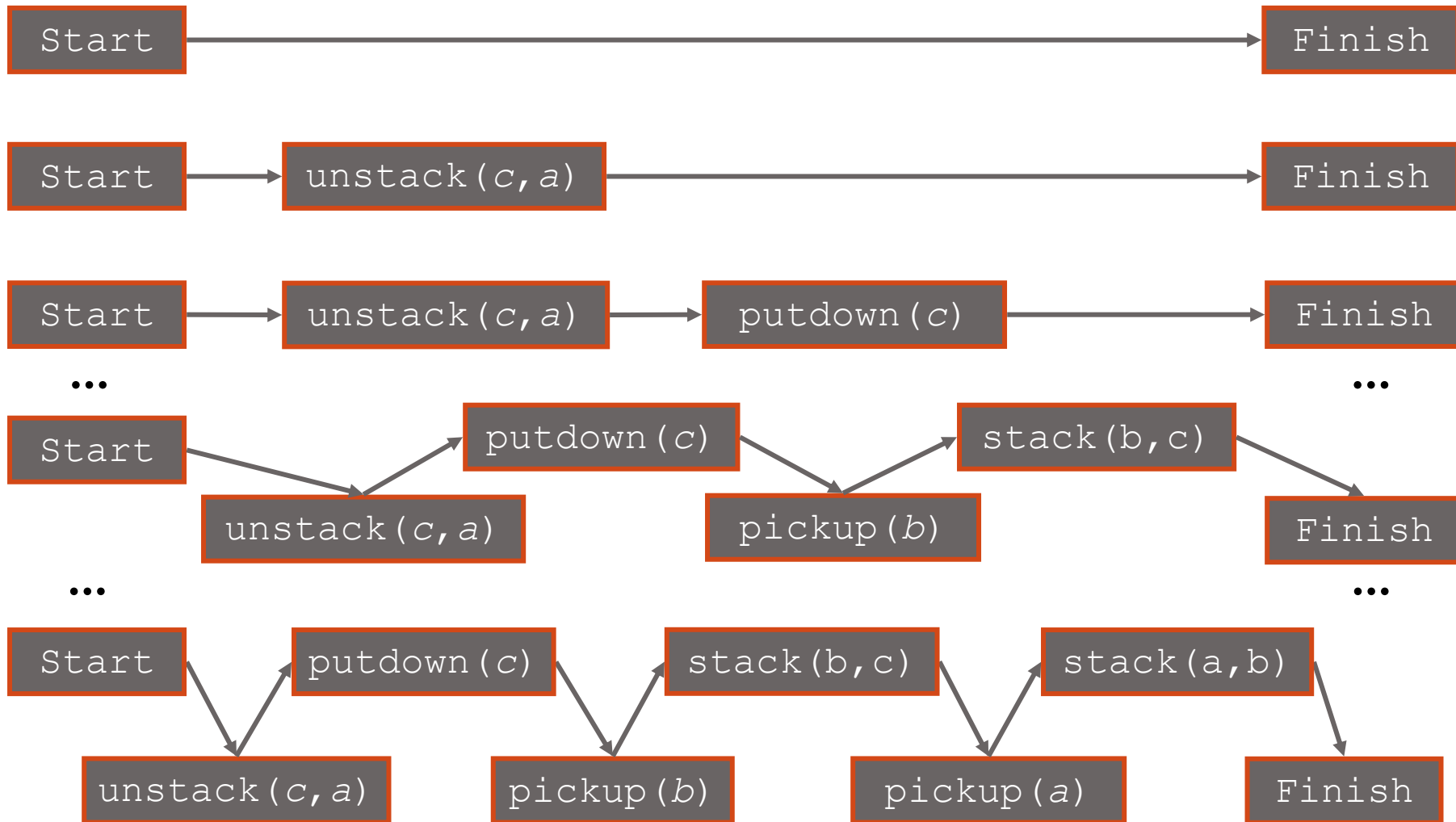
Planning in Blocks World problems



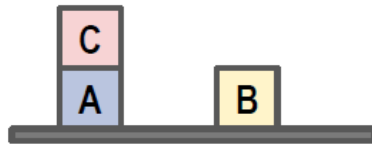
State-Space Search



POP Search Space



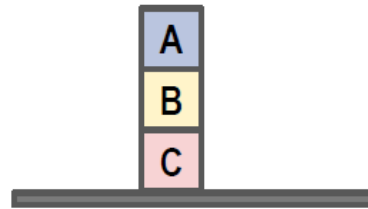
Blocks World



Initial State

Predicates describing
the initial state:

$\text{On}(C, A)$, $\text{On}(A, \text{Table})$,
 $\text{On}(B, \text{Table})$,
 $\text{Clear}(C)$, $\text{Clear}(B)$



Target State

Predicates describing
the target state:

$\text{On}(A, B)$, $\text{On}(B, C)$

ACTIONS:

$\text{Move}(X, Y)$

Precond: $\text{Clear}(X)$, $\text{Clear}(Y)$

Effect: $\text{On}(X, Y)$

$\text{Move}(X, \text{Table})$

Precond: $\text{Clear}(X)$

Effect: $\text{On}(X, \text{Table})$

The planning task is to determine the actions for
reaching the target state from the initial state.

Choosing Actions

ACTIONS:

Move(X, Y)

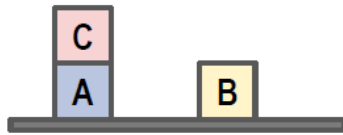
Precond: Clear(X), Clear(Y)

Effect: On(X, Y)

Move(X, Table)

Precond: Clear(X)

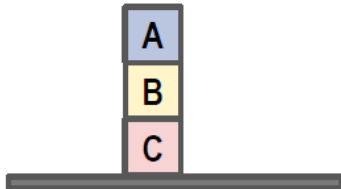
Effect: On(X, Table)



On(C, A), On(A, Table), On(B, Table), Clear(C), Clear(B)

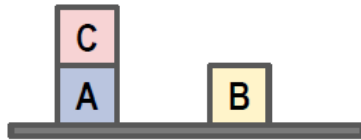


On(A, B), On(B, C)



- We can move C to the table
 - This achieves none of the goal predicates
- We can move C to top of B
 - This achieves none of the goal predicates
- We can move B to top of C
 - This achieves On(B, C)

Partial Solutions

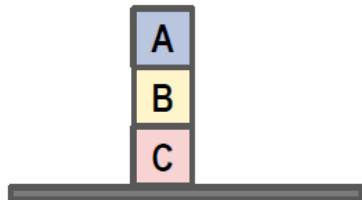


On(C, A), On(A, Table), On(B, Table), Clear(C), Clear(B)

Clear(C), Clear(B)

Move(B, C)

On(A, B), On(B, C)



ACTIONS:

Move(X, Y)

Precond: Clear(X), Clear(Y)

Effect: On(X, Y)

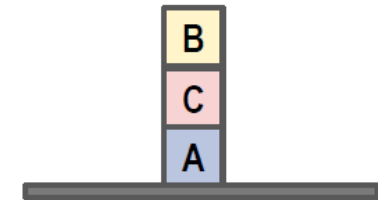
Move(X, Table)

Precond: Clear(X)

Effect: On(X, Table)

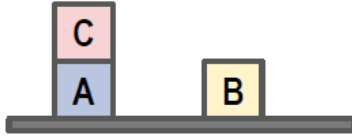
We use Move(B, C) to achieve the sub-goal, On(B, C).

But if we apply this move at the beginning, we get:



Which is not what we want !!

Partial Solutions



On(C, A), On(A, Table), On(B, Table), Clear(C), Clear(B)

Clear(C)

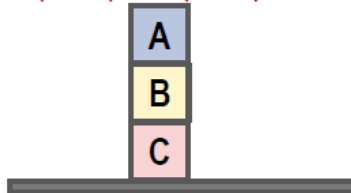
Move(C, Table)

Clear(A), On(C, Table)

Clear(A), Clear(B)

Move(A, B)

On(A, B), On(B, C)



ACTIONS:

Move(X, Y)

Precond: Clear(X), Clear(Y)

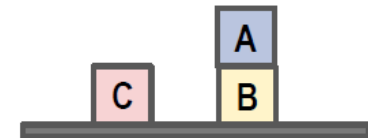
Effect: On(X, Y)

Move(X, Table)

Precond: Clear(X)

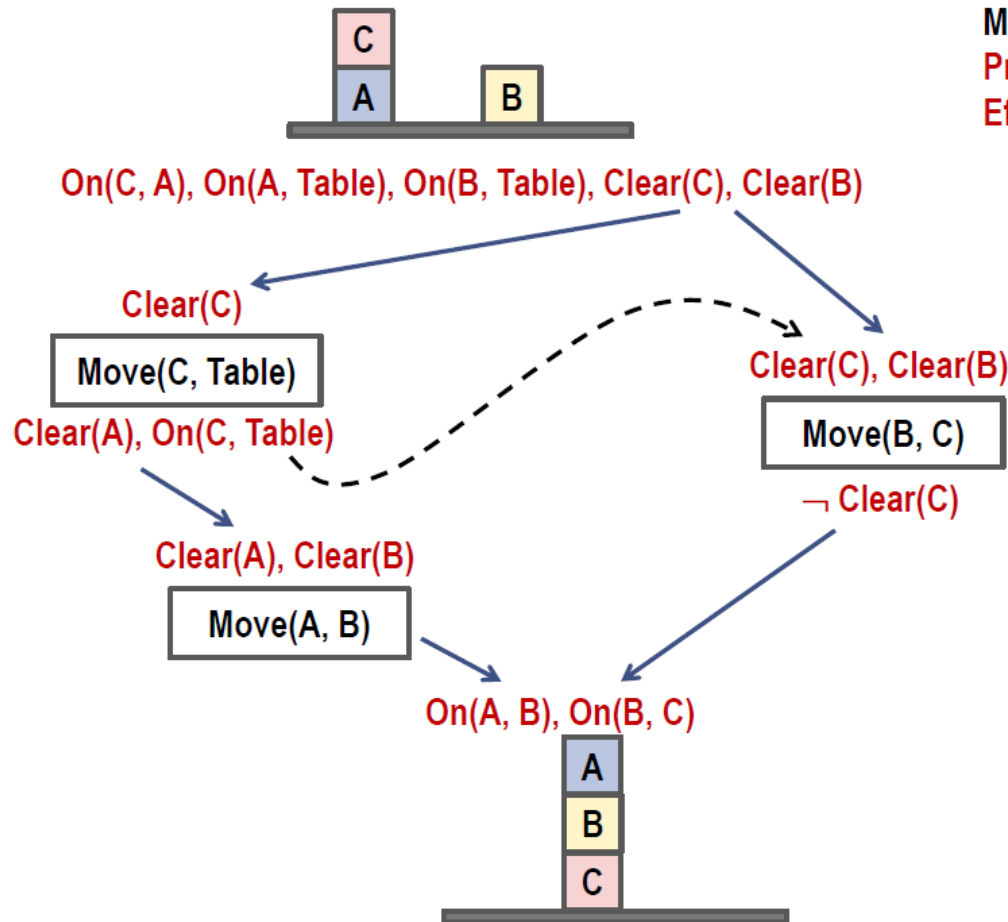
Effect: On(X, Table)

The sub-goal On(A, B) is achieved by moving C to the table and then moving A to top to B. But this gives us:



But this too is not what we want !!

Ordering Partial Solutions



ACTIONS:

Move(X, Y)

Precond: Clear(X), Clear(Y)

Effect: On(X, Y)

Move(X, Table)

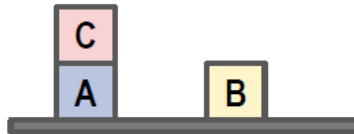
Precond: Clear(X)

Effect: On(X, Table)

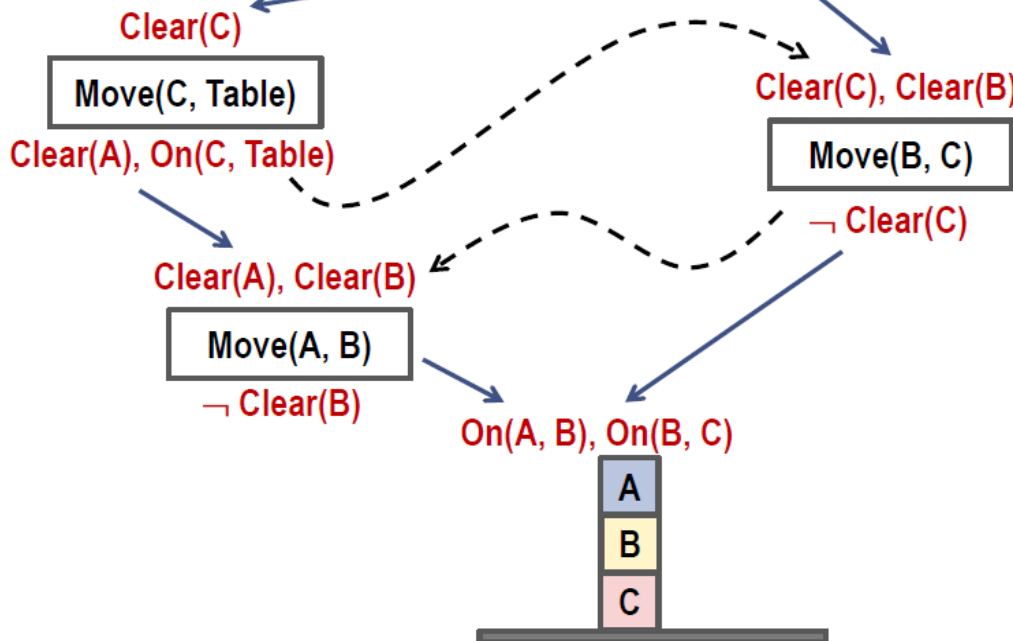
Move(B, C) removes the Clear(C) predicate which is essential for Move(C, Table). Hence Move(C, Table) must precede Move(B, C).

Can Move(B, C) and Move(A, B) be executed in any order?

Ordering Partial Solutions



On(C, A), On(A, Table), On(B, Table), Clear(C), Clear(B)



ACTIONS:

Move(X, Y)

Precond: Clear(X), Clear(Y)

Effect: On(X, Y)

Move(X, Table)

Precond: Clear(X)

Effect: On(X, Table)

Move(A, B) removes the Clear(B) predicate which is essential for Move(B, C). Hence Move(B, C) must precede Move(A, B).

Therefore the only total order is:

1. Move(C, Table)
2. Move(B, C)
3. Move(A, B)

STRIPS

- Stanford Research Institute Problem Solver
- Many planners today use specification languages that are variants of the one used in STRIPS
- Given:
 - **Initial state:** The agent is at *home* without tea, biscuits, book
 - **Goal state:** The agent is at *home* with tea, biscuits, book
 - A set of actions as shown next

Representing States

- States are represented by conjunctions of function-free ground literals

$$\text{At(Home)} \wedge \neg \text{Have(Tea)} \wedge \\ \neg \text{Have(Biscuits)} \wedge \neg \text{Have(Book)}$$

- Goals are also described by conjunctions of literals

$$\text{At(Home)} \wedge \text{Have(Tea)} \wedge \\ \text{Have(Biscuits)} \wedge \text{Have(Book)}$$

- Goals can also contain variables

$$\text{At}(x) \wedge \text{Sells}(x, \text{Tea})$$

- The above goal is *being at a shop that sells tea*

Representing Actions

- Action description – serves as a name
- Precondition – a conjunction of positive literals (why positive?)
- Effect – a conjunction of literals (+ve or –ve)
 - The original version had an *add list* and a *delete list*.

Op(ACTION:	Go(there),
PRECOND:	At(here) $\wedge$ Path(here, there),
EFFECT:	At(there) $\wedge$ $\neg$ At(here))

Representing Plans

- A set of plan steps. Each step is one of the operators for the problem.
- A set of step ordering constraints. Each ordering constraint is of the form $S_i \prec S_j$, indicating S_i must occur sometime before S_j .
- A set of variable binding constraints of the form $v = x$, where v is a variable in some step, and x is either a constant or another variable.
- A set of causal links written as $S \rightarrow c: S'$ indicating S satisfies the precondition c for S' .

Example

- Initial plan

Plan(

STEPS: {

S1: Op(ACTION: start),

S2: Op(ACTION: finish,

PRECOND: RightShoeOn $\wedge$ LeftShoeOn)

},

ORDERINGS: { $S_1 \prec S_2$ },

BINDINGS: {},

LINKS: {})

POP Example: Get Tea, Biscuits, Book

Initial state:

Op(**ACTION:** Start,
 EFFECT: $\text{At}(\text{Home}) \wedge \text{Sells}(\text{BS}, \text{Book})$
 $\wedge \text{Sells}(\text{TS}, \text{Tea})$
 $\wedge \text{Sells}(\text{TS}, \text{Biscuits})$)

Goal state:

Op(**ACTION:** Finish,
 PRECOND: $\text{At}(\text{Home}) \wedge \text{Have}(\text{Tea})$
 $\wedge \text{Have}(\text{Biscuits})$
 $\wedge \text{Have}(\text{Book})$)

Actions:

Op(**ACTION:** Go(y),
 PRECOND: $\text{At}(x)$,
 EFFECT: $\text{At}(y) \wedge \neg \text{At}(x)$)

Op(**ACTION:** Buy(x),
 PRECOND: $\text{At}(y) \wedge \text{Sells}(y, x)$,
 EFFECT: $\text{Have}(x)$)

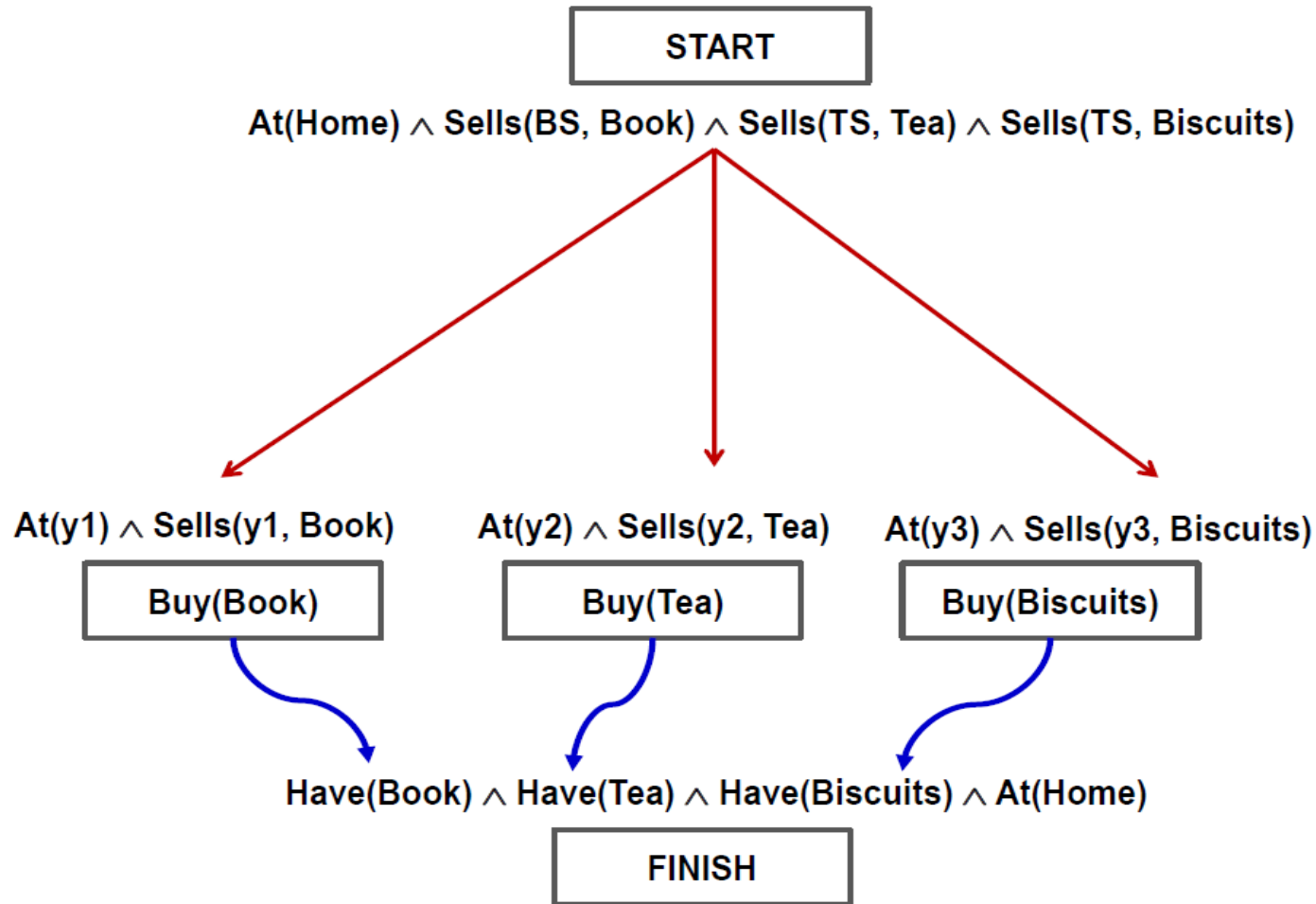
START

$\text{At(Home)} \wedge \text{Sells(BS, Book)} \wedge \text{Sells(TS, Tea)} \wedge \text{Sells(TS, Biscuits)}$

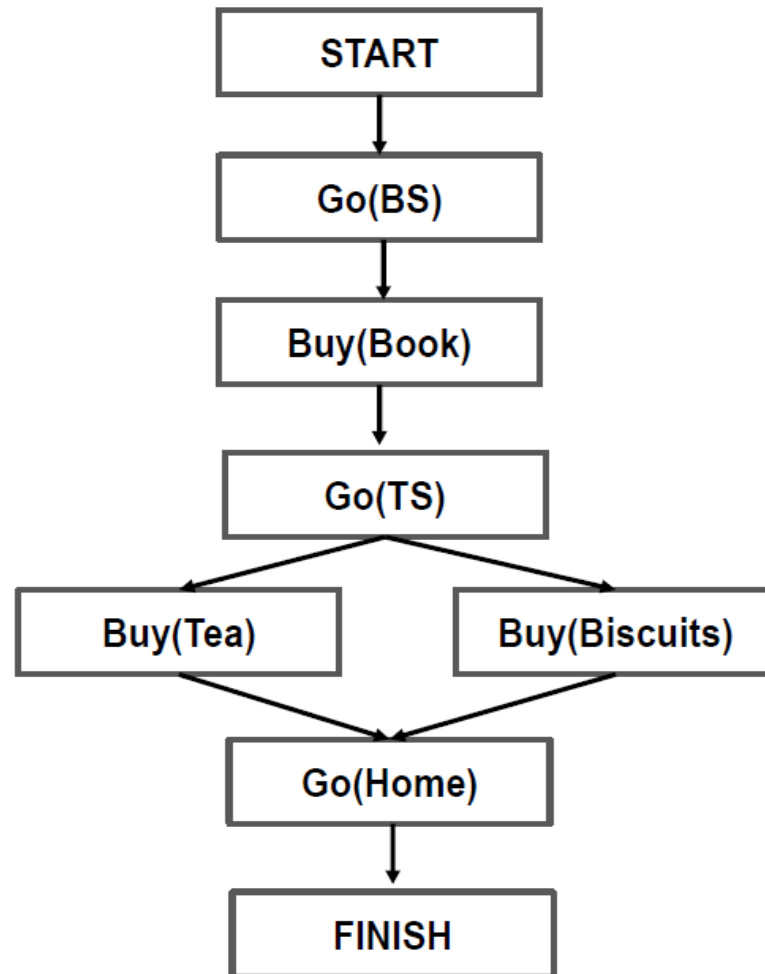


$\text{Have(Book)} \wedge \text{Have(Tea)} \wedge \text{Have(Biscuits)} \wedge \text{At(Home)}$

FINISH



Op(**ACTION:** Buy(x),
PRECOND: At(y) ∧ Sells(y, x),
EFFECT: Have(x))



The Partial Order Planning Algorithm

Function POP(*initial, goal, operators*)

// Returns *plan*

plan ← Make-Minimal-Plan(*initial, goal*)

Loop do

 If Solution(*plan*) then return *plan*

S, c ← Select-Subgoal(*plan*)

 Choose-Operator(*plan, operators, S, c*)

 Resolve-Threats(*plan*)

end

POP: Selecting Sub-Goals

Function Select-Subgoal(*plan*)

// Returns **S**, **c**

 pick a plan step **S** from STEPS(*plan*)

 with a precondition **c** that has not been achieved

Return **S**, **c**

POP: Choosing operators

Procedure Choose-Operator(*plan*, *operators*, *S*, *c*)

Choose a step *S'* from *operators* or *STEPS(plan)* that has *c* as an effect

If there is no such step then **fail**

Add the causal link $S' \rightarrow c: S$ to *LINKS(plan)*

Add the ordering constraint $S' \prec S$ to *ORDERINGS(plan)*

If *S'* is a newly added step from *operators* then add *S'* to *STEPS(plan)* and add $Start \prec S' \prec Finish$ to *ORDERINGS(plan)*

POP: Resolving Threats

Procedure Resolve-Threats(*plan*)

for each **S'** that threatens a link $S_i \rightarrow c: S_j$ in LINKS(*plan*) do
 choose either

Promotion: Add $S' \prec S_i$ to ORDERINGS(*plan*)

Demotion: Add $S_j \prec S'$ to ORDERINGS(*plan*)

if not Consistent(*plan*) then fail

Proc Choose-Operator(*plan*, *operators*, *S*, *c*)

choose a step *S'* from *operators* or STEPS(*plan*) that has *c'* as an effect

such that $u = \text{UNIFY}(c, c', \text{BINDINGS}(plan))$

if there is no such step then fail

add *u* to BINDINGS(*plan*)

add the causal link $S' \rightarrow c: S$ to LINKS(*plan*)

add the ordering constraint $S' \prec S$ to ORDERINGS(*plan*)

if *S'* is a newly added step from *operators* then

add *S'* to STEPS(*plan*) and add $\text{Start} \prec S' \prec \text{Finish}$ to ORDERINGS(*plan*)

Procedure Resolve-Threats(*plan*)

for each $S_i \rightarrow c$: S_j in LINKS(*plan*) do

for each S'' in STEPS(*plan*) do

for each c' in EFFECTS(S'') do

if $\text{SUBST}(\text{BINDINGS}(\textit{plan}), c) = \text{SUBST}(\text{BINDINGS}(\textit{plan}), \neg c')$

then choose either

Promotion: Add $S'' \prec S_i$ to ORDERINGS(*plan*)

Demotion: Add $S_j \prec S''$ to ORDERINGS(*plan*)

if not Consistent(*plan*) then fail

Conditional Planning

Planning environment

- Fully Observable

The agent always knows the current state

- Partially Observable

The agent knows only a certain amount about the actual state.
(common in real world scenarios)

- Unknown

The agent knows nothing about the current state

Classical vs Conditional planning

- Conditional planning (**contingency planning**), is an approach where an agent creates a plan that accounts for multiple possible outcomes or uncertainties in the environment.
- Classical planning assumes a fully deterministic and observable environment.
- Conditional planning addresses situations where the world is only partially observable or actions have uncertain outcomes. The result is a plan that includes conditional branches, specifying different actions depending on what the agent observes at execution time.

Key Characteristics

- **Handling Uncertainty:**

An agent may not know the exact state of the world or the results of its actions beforehand. For example, an action may have several possible outcomes, and the agent must be able to handle any of these outcomes as the plan progresses.

Conditional plans incorporate “if-then-else” structures to deal with uncertainty. Depending on what the agent perceives, it follows different branches of the plan.

IF <condition> Then Plan-A Else Plan-B

- **Sensing Actions:**

An agent needs to gather more information about the environment through sensing. Sensing actions are included in the plan to reduce uncertainty, and the resulting observations determines which branch of the plan to follow.

- **Plan Branches:**

Instead of a linear sequence of actions, a conditional plan consists of a decision tree or directed acyclic graph, where each node represents a state or an action, and edges represent conditional outcomes or observations.

The agent chooses the next action based on the current state and the information gathered from sensing actions or from the results of earlier actions.

- **Goals:**

The ultimate aim is to achieve a goal, therefore, the plan must ensure that the goal is reached regardless of the uncertainties encountered along the way.

Conditional plans aim to cover all possible scenarios that the agent may face, ensuring that the agent can adapt to different situations in real-time.

Steps in Conditional Planning

1. Defining the Initial State and Goal

In conditional planning, the initial state may not be fully known, and the agent needs to gather more information along the way.

2. Identifying Uncertainties

Uncertainties could be the result of actions with probabilistic outcomes, incomplete knowledge of the environment, or dynamic changes during plan execution.

3. Sensing Actions

An agent needs to include sensing actions in the plan to gather information about the current situation.

4. Creating Conditional Branches

Different actions or sub-plans are followed based on the agent's observations.

5. Executing the selected branches

At execution time, the agent follows the plan, making decisions in real-time based on the conditions it encounters.



function AND-OR-GRAPH-SEARCH(*problem*) **returns** *a conditional plan, or failure*
OR-SEARCH(*problem*.INITIAL-STATE, *problem*, [])

function OR-SEARCH(*state*, *problem*, *path*) **returns** *a conditional plan, or failure*
if *problem*.GOAL-TEST(*state*) **then return** the empty plan
if *state* is on *path* **then return** *failure*
for each *action* **in** *problem*.ACTIONS(*state*) **do**
 plan $\leftarrow$ AND-SEARCH(RESULTS(*state*, *action*), *problem*, [*state* | *path*])
 if *plan* $\neq$ *failure* **then return** [*action* | *plan*]
return *failure*

function AND-SEARCH(*states*, *problem*, *path*) **returns** *a conditional plan, or failure*
for each s_i **in** *states* **do**
 plan_i $\leftarrow$ OR-SEARCH(s_i , *problem*, *path*)
 if *plan_i* = *failure* **then return** *failure*
return [**if** s_1 **then** *plan₁* **else if** s_2 **then** *plan₂* **else** ... **if** s_{n-1} **then** *plan_{n-1}* **else** *plan_n*]